

# Chitic David-Alexandru

Systems & Embedded C++ Developer

Bucharest, Romania • [chiticdavid05@gmail.com](mailto:chiticdavid05@gmail.com) • +40 758 031 130  
[github.com/chitic05](https://github.com/chitic05) • [linkedin.com/in/chiticdavid](https://linkedin.com/in/chiticdavid) • [chitic05.github.io/Portfolio](https://chitic05.github.io/Portfolio)

## Summary

Second-year Computer Science student at the University of Bucharest. I like knowing what the machine is actually doing, so most of what I build sits close to the hardware, from ESP32 firmware to tools that work at the level of individual x86 instructions. Looking for an internship in embedded, automotive or low-level security software.

## Education

**University of Bucharest — BSc Computer Science** Sep. 2025 – Jul. 2028 (expected)

*Faculty of Mathematics and Computer Science · Year 2 of 3 · First-year average: 8.50/10*

## Research & Publications

**Extremal Combinatorics — Erdős–Rado Sunflower Problem** Jul. 2026 – Present

*Research group coordinated by Bogdan C. Dumitru (Chief Scientist, Bitdefender)*

[arxiv.org/abs/2609.06175](https://arxiv.org/abs/2609.06175)

- Co-authored *Sunflower-Free Uniform Families: Recursive Constructions and Explicit Bounds* (Sep. 2026), which builds sunflower-free families recursively and derives explicit bounds for small cases.
- I improved both known bounds on  $f(3, 4)$ , the maximum size of a 3-uniform family with no 4-petal sunflower, from  $38 \leq f(3, 4) \leq 69$  to  $39 \leq f(3, 4) \leq 49$ : the lower bound with exhaustive-search programs I wrote, the upper bound with a computer-assisted proof.

## Skills

|                  |                                                                                     |
|------------------|-------------------------------------------------------------------------------------|
| <b>Languages</b> | C++17, C, x86 Assembly, Python, Java                                                |
| <b>Systems</b>   | Linux (Ubuntu), x86 architecture, binary formats, file I/O and page-cache behaviour |
| <b>Embedded</b>  | ESP32 firmware in C++, Bluetooth SPP, servo and motor control                       |
| <b>Design</b>    | Object-oriented design, STL, RAII and smart pointers, data-driven design            |
| <b>Tools</b>     | Git, CMake, SDL3, VS Code (integrated debugging)                                    |
| <b>Spoken</b>    | Romanian (native), English (professional working proficiency)                       |

## Projects

**Bluetooth-Controlled Vehicle** *ESP32, C++, Java (Android)*

[github.com/chitic05/Embedded-Mini-Car](https://github.com/chitic05/Embedded-Mini-Car) • demo: [youtu.be/GVRYfFZE36M](https://youtu.be/GVRYfFZE36M)

- Wrote ESP32 firmware in C++ (Arduino framework) reading newline-delimited text commands over Bluetooth serial into a bounds-checked buffer, driving rear-wheel propulsion and servo-motor steering.
- Built the companion Android application in Java: Bluetooth SPP connection on a background thread with automatic reconnect, and touch controls sending commands in real time.

**MovFuscator — x86 Interpreter & MOV-only Recompiler** *C++, x86 Assembly*

[github.com/chitic05/MovFuscator](https://github.com/chitic05/MovFuscator)

- Wrote a 32-bit x86 interpreter modelling register and stack state, executing instructions step by step and exposing intermediate memory state for inspection.
- Built a partial-evaluation engine rewriting arithmetic and logical instructions into equivalent sequences using only MOV, applying the technique of Christopher Domas's M/o/Vfuscator (a MOV-only C compiler) to individual x86 instructions.

## I/O Benchmark: `mmap` vs. Chunked Reads

*C++, Linux, Windows*

[github.com/chitic05/Benchmarks](https://github.com/chitic05/Benchmarks)

- Measured sequential read throughput on Linux and Windows across two machines: `mmap` and chunked `read()` came within 0.1% of each other on Linux, while chunked reads ran  $1.9\text{--}2.2\times$  faster on Windows — the gap is a platform characteristic, not a universal ranking.
- Found `std::fstream`  $7\text{--}14\times$  slower than either approach; validated every run with checksums across 1- and 5-iteration passes.

## CHIP-8 Virtual Machine Emulator

*C++, SDL3*

[github.com/chitic05/CHIP-8-Emulator](https://github.com/chitic05/CHIP-8-Emulator)

- Implemented the full fetch–decode–execute cycle for all 35 CHIP-8 opcodes over a 4KB address space and 16 8-bit registers.
- Integrated SDL3 for the  $64\times 32$  display, hexadecimal keypad mapping, and delay/sound timers synchronised at a stable 60 Hz.

## Modular 2D Game Engine

*C++, SDL3*

[github.com/chitic05/SDL3-game](https://github.com/chitic05/SDL3-game)

- Data-driven engine loading maps and entity definitions from JSON to fully decouple content from code; resource lifetimes managed with `std::unique_ptr` and a central asset cache.